

BlueBubbles

Revision 1 Structure and Roadmap

Purpose of Revision 1

Build the smallest working version of the LAN messaging application. Two clients should be able to connect to one central Python server, authenticate with simple credentials, and exchange direct text messages. The design is modular from the start so later revisions can replace simple components with PostgreSQL, TLS, encryption, file transfer, groups, auditing and administration.

This revision is intentionally NOT production secure. It is the foundation that later revisions will secure and extend.

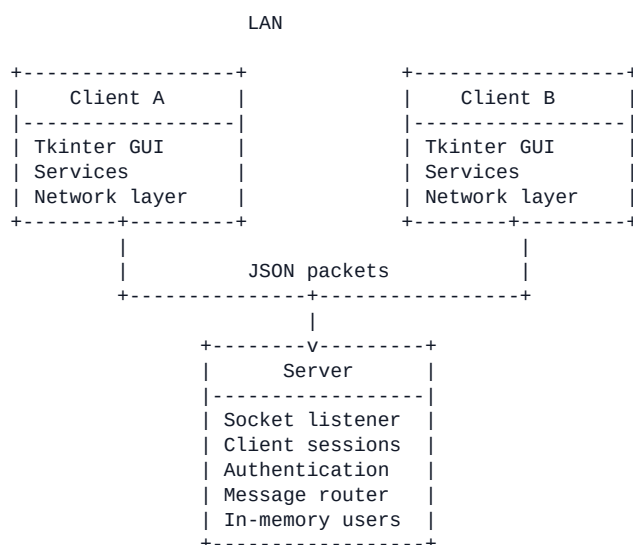
1. Revision 1 Scope

Included: central server, multiple client connections, simple username/password login held in server memory, session state, direct one-to-one text messaging, basic online-user list, JSON-style packets, disconnect/reconnect handling, and a simple Tkinter GUI.

Excluded: PostgreSQL, persistent message history, TLS, application-level encryption, key management, file transfer, groups, audit logs, admin panel, Active Directory, Redis, remote desktop and advanced contact sorting.

The purpose is to prove the client/server architecture before adding complexity. This is consistent with the NEA requirement for modular decomposition into separate areas such as authentication, encryption, messaging, file transfer and auditing.

2. Revision 1 Architecture



The server is the single point through which clients communicate. Clients never open a direct connection to one another. This mirrors the central-storage design in the NEA, while keeping Revision 1 simple enough to implement and test quickly.

3. Exact Revision 1 File Tree

```
BlueBubbles/
|
+-- client/
|   |
|   +-- main.py
|   +-- config.py
|   |
|   +-- network/
|   |   |
|   |   +-- client_connection.py
|   |   +-- message_protocol.py
|   |
|   +-- services/
|   |   |
|   |   +-- authentication_service.py
|   |   +-- messaging_service.py
|   |
|   +-- ui/
|   |   |
|   |   +-- login_window.py
|   |   +-- main_window.py
|   |   +-- chat_window.py
|
+-- server/
|   |
|   +-- main.py
|   +-- config.py
|   |
|   +-- network/
|   |   |
|   |   +-- server.py
|   |   +-- client_session.py
|   |   +-- message_router.py
|   |
|   +-- services/
|   |   |
|   |   +-- authentication_service.py
|   |   +-- messaging_service.py
|
+-- shared/
|   |
|   +-- constants.py
|   +-- enums.py
|   +-- protocol.py
|
+-- tests/
|   |
|   +-- test_protocol.py
|   +-- test_authentication.py
|   +-- test_messaging.py
|
+-- requirements.txt
+-- README.md
```

4. Client Files - Exact Subroutines

client/main.py	
No class	
Subroutine	Purpose
main()	Program entry point; creates and starts the client application.
start_application()	Creates configuration, networking and GUI components, then opens the login window.

client/config.py Class: ClientConfig	
Subroutine	Purpose
__init__()	Stores server IP, server port and basic client settings.
load_config()	Loads configuration values, using defaults when no configuration file exists.
get_server_address()	Returns the server IP/hostname.
get_server_port()	Returns the server port.

client/network/client_connection.py Class: ClientConnection	
Subroutine	Purpose
__init__()	Stores connection settings and the current socket.
connect()	Opens a TCP connection to the server.
disconnect()	Closes the TCP connection cleanly.
is_connected()	Returns whether the client currently has a live connection.
send_request()	Encodes a request and sends it to the server.
receive_response()	Receives and decodes the next server response.
send_packet()	Low-level helper that sends a complete packet through the socket.
receive_packet()	Low-level helper that receives one complete packet.

client/network/message_protocol.py Class: MessageProtocol	
Subroutine	Purpose
create_request()	Creates a standard client-to-server request dictionary.
create_response()	Creates a standard response dictionary for client handling.
encode_packet()	Converts a packet structure into bytes for network transmission.
decode_packet()	Converts received bytes back into a packet structure.
validate_packet()	Checks that a packet contains the required fields.

client/services/authentication_service.py Class: AuthenticationService	
Subroutine	Purpose
__init__()	Stores references to the network connection and current-user state.
login()	Sends credentials to the server and records a successful session.
logout()	Requests logout and clears local session state.
is_logged_in()	Returns whether the client currently has an authenticated user.
get_current_username()	Returns the currently logged-in username.

client/services/messaging_service.py Class: MessagingService	
Subroutine	Purpose
__init__()	Stores the network connection and messaging state.
send_message()	Creates and sends a direct message request.
receive_message()	Processes an incoming message from the server.
get_online_users()	Requests and returns the current online-user list.
poll_messages()	Checks the connection for newly delivered messages.

client/ui/login_window.py Class: LoginWindow	
Subroutine	Purpose
__init__()	Builds the login window and stores required services.
create_widgets()	Creates username, password and login controls.
show()	Displays the login window.
handle_login()	Reads credentials and calls the authentication service.
show_error()	Displays a user-friendly login error.

client/ui/main_window.py Class: MainWindow	
Subroutine	Purpose
__init__()	Builds the main application window.
create_widgets()	Creates the online-user list and main controls.
refresh_users()	Refreshes the list of currently online users.
open_chat()	Opens a chat window for the selected user.
handle_logout()	Logs the user out and returns to the login window.

client/ui/chat_window.py Class: ChatWindow	
Subroutine	Purpose
__init__()	Creates a chat window for one recipient.
create_widgets()	Creates the message display, text input and send button.
display_message()	Adds a sent or received message to the chat display.
send_message()	Reads the input box and asks the messaging service to send it.
clear_input()	Clears the message input box.

5. Server Files - Exact Revision 1 Subroutines

server/main.py No class	
Subroutine	Purpose
main()	Program entry point; creates and starts the server.
start_server()	Creates the server object and begins listening for clients.

server/config.py Class: ServerConfig	
Subroutine	Purpose
__init__()	Stores host, port and development settings.
load_config()	Loads server settings or applies defaults.
get_host()	Returns the server bind address.
get_port()	Returns the listening TCP port.

server/network/server.py Class: Server	
Subroutine	Purpose
__init__()	Stores server settings, socket state and connected sessions.
start()	Creates the listening socket and starts accepting clients.
stop()	Stops accepting connections and closes active sessions.
accept_connections()	Waits for and accepts incoming client connections.
create_client_session()	Creates a ClientSession for a newly connected client.
remove_client_session()	Removes a disconnected session.
broadcast()	Sends a packet to all currently connected clients.
send_to_user()	Sends a packet to a specific logged-in user.

server/network/client_session.py Class: ClientSession	
Subroutine	Purpose
__init__()	Stores one client socket and its authentication state.
run()	Handles requests from this client until disconnect.
send()	Sends a packet to this client.
receive()	Receives a packet from this client.
set_authenticated_user()	Associates the session with a username after successful login.
is_authenticated()	Returns whether the session has successfully logged in.
close()	Closes the client socket.

server/network/message_router.py Class: MessageRouter	
Subroutine	Purpose
__init__()	Stores references to server-side services.
route_request()	Examines the request type and sends it to the correct handler.
route_login()	Handles login requests.
route_logout()	Handles logout requests.
route_message()	Handles direct-message requests.
route_user_list()	Handles requests for the online-user list.

server/services/authentication_service.py Class: AuthenticationService	
Subroutine	Purpose
__init__()	Stores the in-memory user data and session references.
register_user()	Creates a simple development user in memory.
login_user()	Checks credentials and returns a successful login result.
logout_user()	Marks the session as logged out.
user_exists()	Checks whether a username exists.
get_online_users()	Returns usernames of authenticated connected users.

server/services/messaging_service.py Class: MessagingService	
Subroutine	Purpose
__init__()	Stores a reference to active sessions.
send_message()	Validates and forwards a message to its recipient.
deliver_message()	Sends the message packet to a recipient session.
create_message_packet()	Creates the standard server-to-client message packet.

6. Shared Files - Exact Revision 1 Subroutines

shared/constants.py No class	
Subroutine / Item	Purpose
SERVER_HOST	Default server address used during development.
SERVER_PORT	Default TCP port for the messaging server.
BUFFER_SIZE	Maximum size used for one socket receive operation.
MAX_MESSAGE_LENGTH	Development limit for one text message.

shared/enums.py Enums: RequestType, ResponseType	
Subroutine / Item	Purpose
RequestType	Defines LOGIN, LOGOUT, SEND_MESSAGE and GET_USER_LIST request values.
ResponseType	Defines SUCCESS, ERROR and MESSAGE response values.

shared/protocol.py Class: Protocol	
Subroutine / Item	Purpose
create_packet()	Builds a packet with a type and payload.
encode_packet()	Turns a packet into bytes.
decode_packet()	Turns bytes back into a packet.
validate_packet()	Checks that required fields exist and have valid values.
create_success()	Creates a standard successful response.
create_error()	Creates a standard error response.

7. Tests in Revision 1

- **test_protocol.py:** verify packets encode, decode and reject invalid structures.
- **test_authentication.py:** verify valid login succeeds, wrong password fails, unknown user fails, and logout clears authentication state.
- **test_messaging.py:** verify a message from Client A is delivered to Client B and that the server rejects attempts from unauthenticated sessions.

Revision 1 testing should be automated where practical, but the final acceptance test should also involve two real client instances on the LAN.

8. Revision 1 Data Model

Do not create PostgreSQL tables yet. Keep only the minimum development data in server memory:

```
users = {
    "alice": "password1",
    "bob": "password2"
}

sessions = {
    username: ClientSession
}

messages = {
    (sender, recipient): [
        {
            "sender": "alice",
            "recipient": "bob",
            "content": "Hello!"
        }
    ]
}
```

The structure is deliberately temporary. Revision 2+ can move users, sessions and messages to better-defined classes and then persistent storage. No database is required for the Revision 1 goal.

9. Revision 1 Packet Format

```
Login request
{
  "type": "LOGIN",
  "username": "alice",
  "password": "password1"
}

Login response
{
  "type": "SUCCESS",
  "message": "Login successful"
}

Send message
{
  "type": "SEND_MESSAGE",
  "recipient": "bob",
  "content": "Hello Bob"
}

Incoming message
{
  "type": "MESSAGE",
  "sender": "alice",
  "content": "Hello Bob"
}
```

At this stage the packet format is human-readable on purpose. This makes networking bugs much easier for a novice programmer to diagnose. Later revisions can retain the same conceptual protocol while adding authentication tokens, TLS and encrypted payloads.

10. Revision 1 Roadmap - Coding Order

Order	What should work afterwards
Step 1 - Create the project skeleton	Create the folders and empty modules exactly as shown. Add requirements.txt and README.md.
Step 2 - Build the shared protocol	Implement RequestType/ResponseType, packet creation, encoding, decoding and validation. Test this before networking.
Step 3 - Build the server socket	Implement Server, accept TCP clients, create ClientSession objects and log connections to the terminal.
Step 4 - Build the client connection	Implement ClientConnection and prove a client can connect, send one packet and receive one response.
Step 5 - Add request routing	Implement MessageRouter so login and other request types are separated instead of putting all logic in Server.
Step 6 - Add simple authentication	Create AuthenticationService with a small in-memory user dictionary. Make login and logout work.
Step 7 - Connect authentication to the GUI	Create LoginWindow. A correct login opens MainWindow; an incorrect login shows an error.
Step 8 - Add online users	Implement GET_USER_LIST and display connected users in MainWindow.
Step 9 - Add direct messaging	Implement MessagingService on both client and server. Selecting a user opens ChatWindow, typing and sending text delivers it to the recipient.
Step 10 - Add background receiving	Ensure incoming messages are processed without freezing the GUI. Use a simple background thread or polling design.
Step 11 - Add disconnect handling	When a client closes or loses the connection, the server removes its session and remaining clients update their online-user lists.

Order	What should work afterwards
Step 12 - Run the two-client acceptance test	Start one server, connect two clients, log both in, send messages both ways, log out, reconnect and test invalid credentials.

11. What Revision 1 Should Look Like When Finished

1. Start server
 - > server says it is listening
2. Start Client A
 - > login window appears
 - > Alice logs in
 - > main window shows online users
3. Start Client B
 - > Bob logs in
 - > Alice now sees Bob online
4. Alice opens Bob's chat
 - > types: "Hello Bob"
 - > clicks Send
5. Server receives message
 - > router identifies SEND_MESSAGE
 - > messaging service finds Bob's session
 - > server forwards MESSAGE packet
6. Bob receives it
 - > ChatWindow displays "Alice: Hello Bob"
7. Bob replies
 - > Alice receives the response
8. Bob logs out
 - > server invalidates the session
 - > Alice sees Bob go offline

This is the success boundary for Revision 1. Once this works reliably, stop adding features and begin the next revision. Do not add encryption or a database merely because the code appears simple; the point of Revision 1 is to establish a stable modular foundation.

12. Revision 1 Acceptance Checklist

- Server starts without errors and listens on the configured LAN address and port.
- At least two clients can connect at the same time.
- Valid credentials are accepted and invalid credentials are rejected.
- Authenticated clients can see which other users are online.
- One client can send a direct message to another connected client.
- Messages received while the chat window is open appear without restarting the client.
- Logging out removes the session from the server.
- Disconnecting a client does not crash the server.
- Unauthenticated clients cannot send messages.
- The GUI remains responsive while waiting for network data.
- Protocol, authentication and messaging unit tests pass.

13. Deliberate Limitations

Revision 1 is not intended to satisfy the final security criteria. It deliberately leaves plaintext packets, in-memory credentials and non-persistent messages because introducing every final technology at once would make debugging unnecessarily difficult for a novice programmer. The final NEA requirements later require encrypted database messages, encrypted private keys, audit logging, robust recovery and tested encryption functions.

14. The Upgrade Path After Revision 1

Later revision	Purpose
Revision 2	Replace in-memory authentication with PostgreSQL-backed users and proper password hashing.
Revision 3	Persist messages and recover chat history after server restart.
Revision 4	Improve contact list, unread state and general GUI behaviour.
Revision 5	Add groups and membership rules.
Revision 6	Add TLS transport encryption.
Revision 7	Add application-level message encryption and key management.
Revision 8	Add file transfer and checksums.
Revision 9	Add encrypted file transfers.
Revision 10	Add append-only audit logging.
Revision 11	Add server administration and announcements.
Revision 12+	Add search, read receipts, pinned chats, typing indicators, previews and only then consider advanced desired features.

Design note: The strongest Revision 1 implementation is the one you can explain. Keep each module responsible for one concept, keep the protocol simple, and avoid adding technologies that are not yet needed. Your NEA already emphasises abstraction, decomposition and modular design, so this approach gives you a clear foundation for documenting those ideas. ■filecite■turn0file6■L362-L376■